



TP continuación

Pautas:

- **Puede utilizar:**
 - Todo lo visto en la teoría: funciones, algoritmos, estructuras de datos, etc. Salvo librerías de terceros u funciones propias que estén implementadas en el propio lenguaje, a menos que se explice en la consigna.
 - **No es posible utilizar break o continue. Utilizar mejoras por medio de condicionales y cambio de variable.**
 - **No es posible utilizar: Timsort, deque(), queue(), o algún otro algoritmo predefinido por las bondades del lenguaje.**
 - **No se permite el uso de Class para POO en esta etapa.**
 - **Deben resolver los ejercicios.**

Tener en cuenta a la hora de resolver los problemas:

- **Pensamiento abstracto y descomposición**
 - **Abstracción:** capacidad de ignorar detalles irrelevantes y enfocarse en la esencia del problema (ej. ver un "paciente" como un objeto con datos, no como una persona física).
 - **Divide y Vencerás (Top-Down):** habilidad para romper un problema complejo en subproblemas pequeños y manejables (funciones) antes de escribir código.
- **Modularidad:**
 - **Diseño de Módulos:** crear archivos .py independientes que encapsulen funcionalidad específica, permitiendo su reutilización en otros proyectos sin copiar código.
 - **Encapsulamiento:** proteger los datos internos de una estructura y exponer sólo las operaciones necesarias a través de una interfaz pública (funciones).
- **Gestión de datos y estructuras:**
 - **Selección de Estructuras:** saber elegir la herramienta adecuada (Lista, Cola, Pila, Diccionario, Archivo) según las necesidades de acceso, orden y memoria del problema.
 - **Persistencia de Datos:** manipular archivos para que la información sobreviva al cierre del programa(si es condición necesaria), entendiendo modos de apertura y manejo de contextos (with).
 - **Integridad de Datos:** validar entradas y garantizar las reglas de negocio (ej. IDs únicos, estados válidos) se mantengan consistentes.



- **Robustez y calidad de Código:**
 - **Manejo de Excepciones:** evitar errores (archivos faltantes, datos inválidos) y gestionarlos elegantemente sin que el programa colapse.
 - **Eficiencia Algorítmica (Básica):** comprender el impacto de las decisiones de diseño en el rendimiento (evitar bucles innecesarios, entender la diferencia entre $O(1)$, $O(n)$, $O(n^2)$, etc).
- **Resolución de problemas:**
 - **Algoritmos de Ordenamiento y Búsqueda:** implementar o utilizar correctamente métodos para organizar y localizar información eficientemente.
 - **Pensamiento Crítico:** evaluar trade-offs (ej. velocidad vs. memoria, simplicidad vs. escalabilidad) antes de implementar una solución.

1) Se necesita desarrollar un programa informático de atención para el Hospital Regional de la provincia. El hospital atiende diariamente entre 80 y 150 pacientes en su sector de consultorios externos. Actualmente, la gestión de turnos se realiza con fichas de papel, lo que genera desorganización, tiempos de espera excesivos y reclamos de los pacientes.

La dirección del hospital, a través de un consulta de software, encomienda diseñar un sistema en una primera aproximación que gestione el flujo de atención de los pacientes, garantizando que el primer paciente en registrarse sea el primero en ser atendido, sin excepciones.

Reglas de negocio del hospital:

- Orden estricto de atención: los pacientes son atendidos exactamente en el orden en que llegan al sistema. No existe prioridad por tipo de consulta ni por obra social.
- Registro secuencial: cuando un paciente llega, se registra en el sistema y espera su turno.
- Atención por llamado: cuando un médico termina con un paciente, el sistema llama al siguiente paciente en la fila de espera.
- Pacientes urgentes: si un paciente abandona la sala de espera antes de ser atendido, debe ser removido del sistema.
- Consulta de estado: la administración debe poder consultar si un paciente específico está esperando y qué posición aproximada ocupa.
- Cierre diario: al finalizar el día, se debe generar un reporte ordenado de todos los pacientes que fueron atendidos, ordenados por obra social (para facturación), con todos los datos completos.



Cada paciente registrado debe contener:

- dni: Documento Nacional de Identidad (único).
- nombre_completo: nombre y apellido.
- obra_social: obra social del paciente.
- consultorio: consultorio asignado (ej: "C-101").
- motivo_consulta: razón de la visita.
- hora_llega: hora de registro (HH:MM).
- estado: "Esperando" o "Atendido".

A su vez, nos comenta que existen determinadas restricciones técnicas a tener en cuenta:

- Escalabilidad: el sistema debe manejar al menos 200 pacientes diarios sin degradación.
- Integridad: un paciente no puede registrarse dos veces con el mismo DNI mientras esté en espera.
- Persistencia: al cerrar el día, los datos deben guardarse en un archivo para historial.
- Reporte ordenado: el archivo de cierre debe estar ordenado por obra social (no por orden de llegada).
- Módulo independiente: la lógica de gestión debe estar separada del programa principal para poder reutilizarla en otros sectores del hospital.

Analizar y tener en cuenta para el diseño y codificación de la solución:

- Si llegan los pacientes en este orden: Pérez (08:01), Gómez (08:05), Ruiz (08:12), ¿en qué orden deben ser atendidos?, ¿Qué principio matemático describe este comportamiento?
- ¿Qué estructura de datos de refleja naturalmente este comportamiento?, ¿por qué?
- Si necesitan buscar si el paciente con DNI "30123456" está esperando, ¿qué operación necesitas?, ¿cuál sería la complejidad?
- Al final del día, si se necesita un reporte ordenado por obra social y no por orden de llegada, ¿necesita la misma estructura para el reporte o una diferente? ¿Por qué?

Tener en cuenta que nos piden desarrollar como modulo `gestion_turnos.py` los cuales deben poder funcionar como:

- `registrar_paciente(...)`: agrega un paciente al final de la espera
- `llamar_siguiente(...)`: atiende al paciente que lleva más tiempo esperando



- `buscar_paciente(...)`: consulta si un paciente está esperando
- `remover_paciente(...)`: elimina un paciente que abandonó la espera
- `pacientes_en_espera(...)`: cantidad de pacientes esperando
- `generar_reporte_ordenado(...)`: lista ordenada por campo especificado
- `exportar_reporte(...)`: persiste el reporte en archivo CSV
- `cargar_historial(...)`: carga datos desde archivo CSV

Requisitos de Código:

- Manejo de excepciones personalizadas: "PacienteDuplicado", "Paciente No Encontrado", "Espera Vacía".
- Validación de datos (DNI único, campos no vacíos).
- Documentación de complejidad, respetar Big O .
- Uso de `with` para manejo de archivos.
- Código modular: separar lógica de presentación.

Comportamiento esperado:

GESTIÓN DE TURNOS

[1] REGISTRANDO PACIENTES...

María Pérez (DNI: 28123456) registrado. Posición en espera: 1
Juan Gómez (DNI: 30456789) registrado. Posición en espera: 2
Laura Ruiz (DNI: 25789012) registrado. Posición en espera: 3
Carlos Díaz (DNI: 32345678) registrado. Posición en espera: 4
Ana Martínez (DNI: 29876543) registrado. Posición en espera: 5
Pedro López (DNI: 35123456) registrado. Posición en espera: 6
Sofía Fernández (DNI: 26543210) registrado. Posición en espera: 7

[2] ATENDIENDO PACIENTES...

Llamando al siguiente paciente: María Pérez (DNI: 28123456)
Llamando al siguiente paciente: Juan Gómez (DNI: 30456789)

[3] CONSULTA DE ESTADO...

Paciente DNI 32345678: Carlos Díaz - Estado: Esperando
Paciente DNI 99999999: No encontrado en espera

[4] PACIENTE ABANDONA LA SALA...

Laura Ruiz (DNI: 25789012) removido de la espera

[5] CONTINUAR ATENCIÓN...

Llamando al siguiente paciente: Carlos Díaz (DNI: 32345678)



Llamando al siguiente paciente: Ana Martínez (DNI: 29876543)

[6] GENERANDO REPORTE DIARIO (ordenado por obra social)...

Exportando reporte a: reporte_diario_AÑO_MES_DIA.csv

- Pacientes atendidos hoy: 4
- Pacientes aún en espera: 2

[7] AL UTILIZAR EL SAVE PERMITE GUARDAR LO DEL DIA.

Reporte guardado exitosamente.

Elegir, desarrollar el código en Python y justificar que estructura de datos permite resolver dicha problemática. Respetar las restricciones técnicas y los tiempos de BigO.

2) En una empresa startup de satélites están iniciando la toma de datos para poder implementar los diferentes sistemas de monitoreo y control y el cliente “**Centro de Control Satelital Nacional**” (**CCSN**) pide con urgencia poder diseñar el sistema de gestión de datos orbitales que permitirá monitorear, controlar y analizar la flota de satélites del país.

El sistema debe cumplir con las siguientes funcionalidades:

- **Registro de satélites:** agregar nuevos satélites con código único y múltiples atributos.
- **Consulta por código:** buscar un satélite específico por su identificador único.
- **Listado completo:** mostrar todos los satélites ordenados por código.
- **Eliminación:** remover satélites dados de baja o desactivado.
- **Exportación CSV:** generar archivo con todos los satélites en ese momento.
- **Consolidación:** fusionar múltiples archivos de carga en un solo archivo maestro. Los archivos de carga están desordenados. Pueden ser de a 3 al mismo tiempo. La consolidación se debe realizar recorriendo una sola vez para generar el maestro.
- **Estadísticas globales, calcular totales:** distancia recorrida, errores acumulados, sensores activos.

Cada satélite debe almacenar la siguiente información:

- Código: identificador único (ej: "SAT-ARG-001").
- Nombre: nombre oficial del satélite.



- Recorrido_km: distancia total recorrida en órbita (km).
- Fecha_lanzamiento: fecha de lanzamiento (YYYY-MM-DD).
- Pais_origen: país fabricante.
- Funcion: función principal(Observación, Comunicaciones, Meteorología, etc.).
- Tipo: tipo de órbita (LEO, MEO, GEO, Polar).
- Imagenes_descargadas: cantidad total de imágenes procesadas.
- Errores_detectados: cantidad de errores registrados.
- Cantidad_sensores: número de sensores instalados.
- Resolucion_espacial: resolución en metros/píxel.
- Resolucion_espectral: resolución espectral (bandas).
- Estado: "activo" o "inactivo".

Restricciones Técnicas:

- **Datos en memoria:** el sistema opera en memoria RAM durante la sesión y con persistencia en archivos.
- **Escalabilidad:** debe manejar al menos 500 satélites sin degradación significativa. aunque la cantidad puede aumentar con el tiempo.
- **Integridad:** el código de satélite debe ser único (no se permiten duplicados).
- **Ordenamiento:** el archivo maestro debe estar siempre ordenado por código.
- **Recursos limitados:** el servidor tiene 8GB de RAM disponibles para el sistema.
- **Persistencia:** los datos deben sobrevivir al reinicio del sistema mediante archivos.

La implementación del modulo que se llama gestion_satelites.py debe contener las siguientes funcionalidades:

- agregar_satelite(...) con tiempo $O(n)$ o $O(1)$
- buscar_por_codigo(...) con tiempo $O(n)$ o $O(1)$
- eliminar_satelite(...) con tiempo $O(n)$ o $O(1)$
- ordenar_por_codigo(datos) con tiempo a elección, en esta primera etapa.
- exportar_csv(...) con tiempo $O(n)$
- cargar_desde_csv(ruta_archivo) con tiempo $O(n)$
- consolidar_archivos(listas_archivos) con tiempo a elección, en esta primera etapa.
- generar_estadisticas(datos) con tiempo $O(n)$.



Requisitos de Código:

- Manejo de excepciones personalizadas con mensajes => (Satélite Duplicado, Satélite No Encontrado)
- Validación de datos (código único, valores numéricos positivos donde corresponda)
- Documentación de complejidad.
- Uso de with para manejo de archivos.
- Código modular: separar lógica de presentación

Datos de ejemplo:

satelites_muestra.csv (Archivo 1 - Desordenado)

```
SAT-ARG-003,Orion-3,125000.50,2022-03-15,Argentina,Observación,LEO,15000,12,8,2.5,4,Activo
SAT-ARG-001,Aurora-1,89000.25,2021-06-10,Argentina,Meteorología,Polar,22000,5,12,1.0,8,Activo
SAT-ARG-005,Nebula-5,200000.00,2023-01-20,Brasil,Comunicaciones,GEO,8000,30,6,5.0,2,Inactivo
```

satelites_muestra2.csv (Archivo 2 - Desordenado)

```
SAT-ARG-002,Cosmos-2,150000.75,2021-11-05,Argentina,Observación,LEO,18000,8,10,1.5,6,Activo
SAT-ARG-004,Tierra-4,95000.00,2022-08-30,Chile,Meteorología,Polar,12000,15,9,2.0,5,Activo
```

Caso de Consolidación Esperado

Al consolidar los dos archivos anteriores, el archivo maestro debe quedar así:

```
SAT-ARG-001,Aurora-1,89000.25,2021-06-10,Argentina,Meteorología,Polar,22000,5,12,1.0,8,Activo,89000.25
SAT-ARG-002,Cosmos-2,150000.75,2021-11-05,Argentina,Observación,LEO,18000,8,10,1.5,6,Activo,150000.75
SAT-ARG-003,Orion-3,125000.50,2022-03-15,Argentina,Observación,LEO,15000,12,8,2.5,4,Activo,125000.50
SAT-ARG-004,Tierra-4,95000.00,2022-08-30,Chile,Meteorología,Polar,12000,15,9,2.0,5,Activo,95000.00
SAT-ARG-005,Nebula-5,200000.00,2023-01-20,Brasil,Comunicaciones,GEO,8000,30,6,5.0,2,Inactivo,200000.00
```

Estadísticas Globales Esperadas:

```
Total Satélites: 5
Distancia Total Recorrida: 659001.50 km
Errores Totales: 70
Sensores Activos: 45
Satélites Activos: 4
Satélites Inactivos: 1
```



A nivel de código se espera:

```
from gestion_satelites import (  
    cargar_desde_csv,  
    agregar_satelite,  
    buscar_por_codigo,  
    eliminar_satelite,  
    consolidar_archivos,  
    generar_estadisticas,  
    exportar_csv  
)
```

y que, por ejemplo, para consolidar archivos pueda operar:

```
archivos_entrada = ["satelites_lote1.csv", "satelites_lote2.csv", "satelites_lote3.csv"]  
  
archivos_consolidados = consolidar_archivos(archivos_entrada)  
  
etc.
```

Elegir, desarrollar el código en Python y justificar qué estructura de datos permite resolver dicha problemática.

Respetar las restricciones técnicas y los tiempos de BigO.

3) Nos encargan como desarrollador poder elaborar un sistema de punto de venta (POS) para una cadena de comercios con cantidad finita de al menos 100-200 comercios. El sistema debe gestionar una base de datos de clientes en memoria, por lo tanto, nos encomiendan utilizar alguna estructura de datos que permita gestionar los clientes con sus atributos. Tener en cuenta que son varios clientes con atributos los cuales nos dan y cada cliente requiere ser indexado simultáneamente por dos coordenadas independientes para localizarlo de manera única, dentro del espacio de datos. También hay que tener en cuenta que, a su vez, las funciones que se necesitan deben ser utilizadas por un módulo portable que permita poder ser utilizados con el resto del sistema. Además, requiere tener un tiempo de acceso de $O(1)$, en el mejor caso para acceder al cliente.

El cliente tendrá:

- ID_Cliente (único).
- Nombre.



- Email.
- Saldo.
- Estado ("Activo" o "Inactivo").

Requerimientos del Módulo:

- `agregar_cliente(...)`:
 - Verificar que el ID no exista previamente. Si existe, lanzar una excepción personalizada "ClienteDuplicado".
 - ¿Cómo afecta esto a la complejidad temporal si la matriz crece a 100,000 clientes?
- `eliminar_cliente(...)`:
 - Si no existe, lanzar `ClienteNoEncontrado`.
 - Al eliminar un cliente ¿Cómo manejas el desplazamiento de memoria?
- `ordenar_por_saldo(...)`:
 - Ordena in-place (sin crear una nueva DB) basándose en el Saldo.
- `buscar_cliente_rapido(...)`:
 - si la DB está ordenada por ID (hipotéticamente), ¿podrías usar búsqueda binaria? Como no lo está, ¿cuál es el default?
- `generar_reporte_resumen(...)`:
 - El reporte debe estar en un archivo de texto: `reporte.txt`. En donde esté presente: `total_clientes`, `saldo_total`, `clientes_activos`.
 - El reporte debe iterar una sola vez la DB.
- Código modular: separar lógica de presentación

Tendría que funcionar de la siguiente manera:

```
from gestion_clientes import EstructuraClientes, ver_reporte

sistema = EstructuraClientes()

sistema.agregar_cliente(...)

sistema.agregar_cliente(...)

....

path_file_reporte = sistema.generar_reporte()

ver_reporte(path_file_reporte)
```



Elegir, desarrollar el código en Python y justificar qué estructura de datos permite resolver dicha problemática. Recordar que los datos con los cuales se trabaja son finitos y conocidos.

Nota a tener en cuenta:

El uso de la IA es válido como asistente en el proceso de investigación; sin embargo, se espera que demuestren su propia comprensión mediante la revisión y el análisis profundo del contenido que ustedes vayan redactando y asimilando.

Asimismo, es indispensable adaptar cualquier información a las normas APA vigentes, asegurando que las referencias sean verificables y estén correctamente formateadas.

Se tendrá en cuenta también:

- Coherencia y estructura de lo que entregue.
- Originalidad de la redacción.